

MARSHALLING OF DATA BETWEEN FORTRAN AND VISUAL BASIC**REVISION 0, 06/21/26**

Introduction

Marshalling is the process of transforming memory representations of data objects into a compatible, standardized format—such as a stream of bytes or a JSON string—so they can be safely transmitted, stored, or reconstructed by a different programming language. This document covers the data types for the JSON standard against the data types used by Fortran. Following that is a discussion of how data is ‘marshalled’ as it flows from the client (VB.NET) to the server (Fortran) with JSON RPC handling the transfers. Last but not least, two open source libraries, *StreamJsonRPC* (for .NET) and *json-fortran* (for Fortran) are discussed.

Refer to the document “Fortran Interoperability: ISO_C_BINDING vs. Windows Named Pipes + JSON-RPC 2.0” for a comparison of the advantages and disadvantages of the use of JSON RPC versus ISO_C_BINDING for the transfer of data between languages.

Comparison of JSON and Fortran data types

Below is a comparison of **JSON data types** and **Fortran intrinsic types**, showing where they match cleanly and where they do not.

? JSON vs. Fortran Data Types (Simple Comparison)

JSON Type	Fortran Type	Match?	Notes
String	CHARACTER	Partial	JSON strings are variable-length Unicode; Fortran strings are fixed-length unless allocatable.
Number	INTEGER / REAL	Partial	JSON has one “number” type; Fortran distinguishes integer, real, and precision kinds.
Boolean	LOGICAL	Yes	Direct conceptual match.
n/a	COMPLEX	No	No native support for complex numbers in JSON RPC
Null	(none)	No	Fortran has no null value for scalars; closest analog is optional arguments or pointers.
Array	ARRAY	Partial	JSON arrays are heterogeneous; Fortran arrays must be homogeneous and typed.

JSON Type	Fortran Type	Match?	Notes
Object	Derived Type	Partial	JSON objects are dynamic key–value maps; Fortran derived types are static and strongly typed.

Notes:

- JSON is **dynamic, loosely typed, and Unicode-based**.
- Fortran is **static, strongly typed, and uses fixed-layout data structures**.
- Numeric and logical values map cleanly; complex numbers, strings, arrays, and objects require **marshalling or conversion logic**.
- Note that VB.NET has native support for complex numbers, but JSON does not.

How does the Fortran server parse and convert values arriving from the VB.NET client program?

The Fortran static library uses the *json-fortran* library to deserialise the inbound JSON body into an in-memory tree of `json_value` nodes. When the dispatch engine calls a method handler, it passes a pointer to the `params` subtree of that tree rather than a raw string, so by the time handler code runs the JSON has already been fully parsed. Handler subroutines then call the library's helper routines (`jsonrpc_get_int_param`, `jsonrpc_get_string_param`, and their named-parameter equivalents) which call *json-fortran's* `json%get()` to extract individual values; *json-fortran* performs the type coercion internally, converting a JSON number to a Fortran `INTEGER` or `REAL(8)` and a JSON string to a Fortran `CHARACTER` allocatable. JSON booleans (`true/false`) map to Fortran `LOGICAL` through the same mechanism. For named-parameter calls such as matrix operations, handlers extract each field from the `params` object by name, validate the reported type string against an expected value, and then extract the data array element by element. If extraction fails at any point the handler sets an error code of `-32602` and returns without performing the computation.

How does the VB.NET client parse and convert values arriving from the Fortran server?

The VB.NET client delegates almost all deserialisation to the *StreamJsonRpc* library, which uses the `System.Text.Json` / `Newtonsoft.Json` stack underneath. Because every outbound call is made through a generic `InvokeAsync(Of T)` wrapper, the expected .NET return type `T` is baked into the call at compile time; *StreamJsonRpc* uses that type token to instruct the JSON deserialiser how to map the incoming `result` field — a JSON integer becomes a .NET `Integer`, a JSON floating-point number becomes a `Double`, and a JSON string becomes a `String`, all without any explicit parsing code in the application. For named-parameter calls that return a complex object (such as a matrix result), the result arrives as a JSON object and *StreamJsonRpc*

deserialises it into a .NET `Object` or `Dictionary`; the calling application then casts or unpacks the fields it needs. The `ValidationModule` in the DLL provides utility helpers for verifying that numeric results fall within expected tolerances, but the primary type conversion is entirely handled by the framework before those helpers are called.

How do both sides handle complex numbers, given JSON has no native complex type?

Because the JSON-RPC 2.0 specification has no complex number type and *json-fortran* likewise has no complex value kind, the system uses a canonical string encoding as a lingua franca: a complex number is always transmitted as a string in the form `re±imi` (for example `"3.5+2.1i"` or `"1.0-4.0i"`). On the client side, before making a complex arithmetic call, the VB.NET application manually formats both operands into that string representation and passes them to the `ExecuteStringOperationAsync` wrapper, which sends them to the server as ordinary JSON strings. The Fortran handler receives two `CHARACTER` parameters, parses each string by locating the `±` separator and the trailing `i` character, converts the substrings to `REAL(8)` values using Fortran's internal `read`, performs the arithmetic on native Fortran `COMPLEX` values (using the conjugate-of-denominator formula for division), and then formats the result back into the same canonical string. The server returns that string in the JSON response `result` field, *StreamJsonRpc* deserialises it as a plain `String` on the VB.NET side, and the client application then parses the string back into real and imaginary components for display and verification. The encoding and decoding of complex values is therefore entirely the responsibility of the two applications; the DLL and the static library are unaware that strings with this particular format represent complex numbers.

Existing JSON libraries for VB.NET and Fortran

Two open source libraries, *StreamJsonRpc* for .NET languages, and *json-fortran* for Fortran programs provide most of the support for the implementation of JSON RPC 2.0 standard for this project. Below is a quick summary of each library.

StreamJsonRpc

StreamJsonRpc is an open-source .NET library developed and maintained by Microsoft that implements the JSON-RPC 2.0 protocol over any bidirectional stream — including named pipes, TCP sockets, WebSockets, or stdin/stdout. It handles the full JSON-RPC lifecycle: message framing, serialization/deserialization, method dispatch, request/response correlation by `id`, batch requests, progress notifications, and cancellation tokens. The library supports both `System.Text.Json` and `Newtonsoft.Json` as the underlying serializer, and exposes a clean .NET API where the user will annotate ordinary VB.NET (or C#, F#) methods with attributes and the library automatically routes incoming JSON-RPC calls to them — eliminating the need to hand-write a message loop or parser.

From a practical standpoint, *StreamJsonRpc* is particularly well-suited as the .NET-side counterpart to a Fortran named-pipe server. A VB.NET client can connect to a Fortran-created named pipe with a few lines of code, wrap it in a `JsonRpc` instance, and invoke remote methods

as if they were ordinary async .NET method calls — with full support for `CancellationToken`, strongly-typed return values, and automatic exception propagation across the pipe boundary. It is the same library that powers Visual Studio's out-of-process language server communication and the Live Share extension, so it is production-hardened at scale. The main constraint is that it is .NET-only on the client side; the Fortran server must implement the JSON-RPC 2.0 wire format independently, typically via a library like *json-fortran* (and the static library provided by this project). It should be noted that `StreamJsonRpc` library does the heavy lifting of implementing the JSON RPC protocol for the client; the DLL provided is simply a collection of helper functions that may speed up development of .NET client programs.

json-fortran

json-fortran is an open-source, pure-Fortran library (available on GitHub at [jacobwilliams/json-fortran](https://github.com/jacobwilliams/json-fortran)) that provides full JSON reading, writing, and manipulation capabilities for modern Fortran (Fortran 2008 and later). It is built around a tree-structured in-memory representation of JSON data using Fortran derived types and linked lists, where a `json_file` object acts as the top-level handle and `json_value` nodes represent the individual elements of the JSON tree — objects, arrays, strings, numbers, booleans, and nulls. The library supports both reading JSON from files or strings and writing/serializing Fortran data back out to JSON, with a straightforward API of `get`, `add`, `update`, and `print` style procedures. It handles Unicode (via `ISO_10646` kind characters), supports both `real32` and `real64` floating-point kinds, and can be built with the Intel Fortran compiler IFX in Visual Studio, CMake or FPM (Fortran Package Manager), making it relatively easy to integrate into modern Fortran build systems.

From a practical standpoint, *json-fortran* is the most mature and actively maintained JSON library available in the Fortran ecosystem, and it would be the natural choice for implementing the Fortran server side of a Named Pipes + JSON-RPC 2.0 architecture. However, it does have limitations worth noting: it is not a JSON-RPC implementation — it only handles JSON serialization and parsing, meaning the developer must still hand-write the JSON-RPC 2.0 message framing, the request dispatch loop, the `id` correlation logic, and the named pipe I/O in Fortran (which the static library in this project provides). Performance on very large arrays is also a consideration, as the tree-based in-memory model carries more overhead than a streaming parser would. Nevertheless, for the level of data complexity typical in a Fortran-to-.NET service interface, *json-fortran* is well up to the task and is substantially more robust than attempting to hand-roll JSON parsing in Fortran from scratch.